DETAILED DESIGN DOCUMENT

REVA Internship Opportunities (RIO) | School of Computer Science and Engineering, REVA University, Bengaluru

Particulars	Details
Task	Task 4 (SL 4), Project Track: Detailed Design of the Module
Name of Student	R Sai Pranav
University Seat Number (USN)	R23EM069
Project Title	ForgeGuard: An AI-Based Document Forgery Detection and Verification System
Track	PS 109
Faculty Guide	Mala B M
Date of Submission	2 November 2026

1. Introduction

1.1 Purpose. The System Design Document fixed the architecture: four detector services behind one orchestrator, one API, one data model. This document goes one level deeper, into the classes, algorithms, data structures and interfaces that implement that architecture, so the design is detailed enough to code (or, for the assigned module, already has been). Identifiers carry over unchanged from the Literature Survey (Gap N), Project Objectives (O1–O9) and System Design Document (FR/NFR) so all four documents trace against each other.

1.2 Scope. Section 2 gives full detailed design — classes, algorithms, interfaces, error handling — for the assigned module, Document Tamper Detection and Localisation, matching the working prototype in `tamper_localisation/`. Section 3 gives the same depth of detail for the remaining three services and the orchestrator at the algorithm/pseudocode level, since they are designed but not yet implemented. Section 4 gives a message-level sequence diagram for one verification request. Section 5 covers system-wide error handling. Section 6 is the full traceability table for this document.

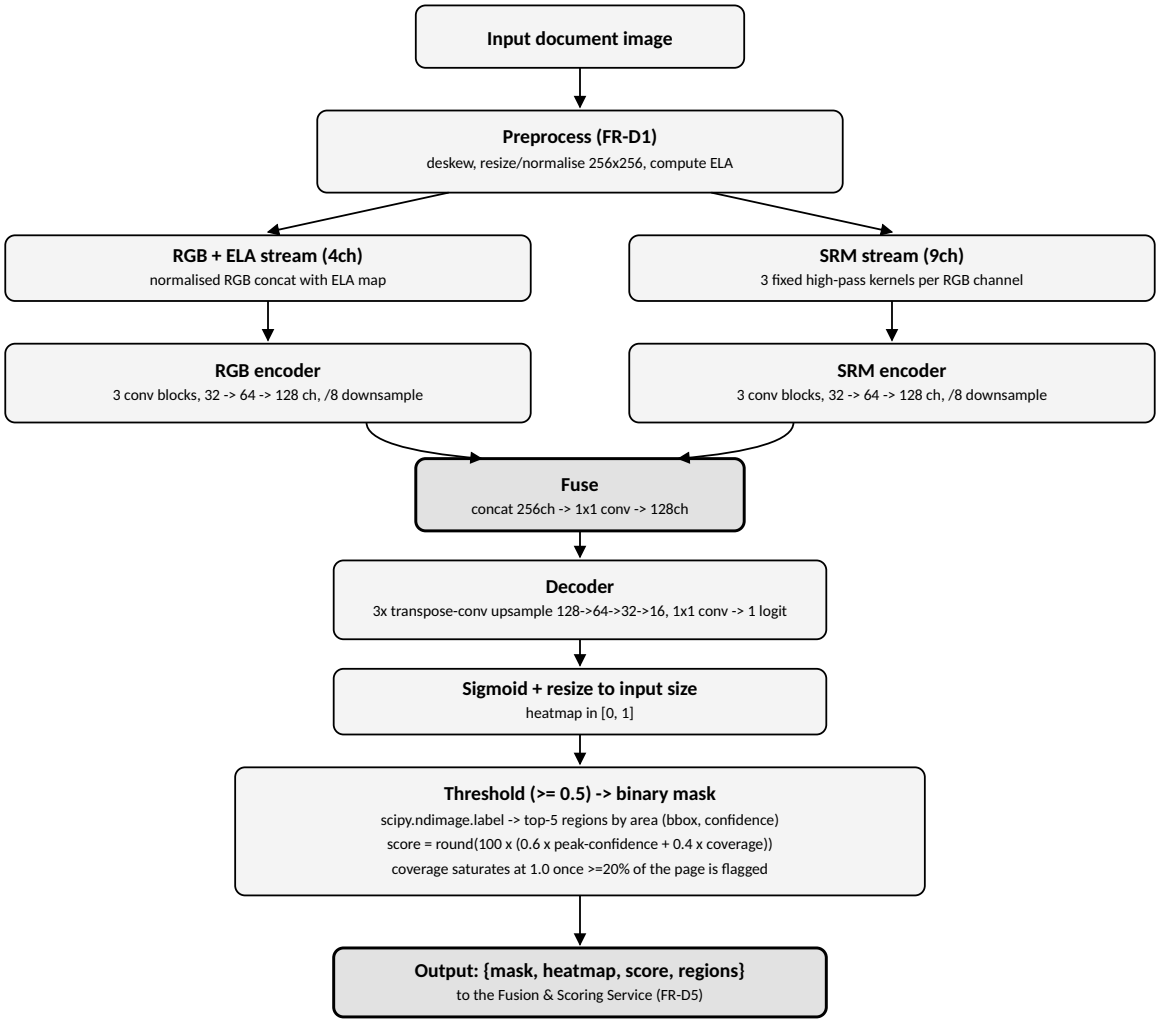
2. Detailed Design — Tamper Detection and Localisation (assigned module)

2.1 Component breakdown

Component	Responsibility
`SRMFilter`	Fixed, non-trainable depthwise convolution applying 3 high-pass kernels (horizontal residual, vertical residual, Laplacian-style) to

	each input channel. 3-channel RGB in → 9-channel noise-residual tensor out. Implements the noise-residual half of FR-D1/FR-D2.
`preprocess()`	FR-D1: deskew (row-variance search over ±10°), resize/normalise to 256×256 in [0,1], compute the ELA map (JPEG re-save at quality 90, pixel-wise difference). Returns the RGB tensor and the single-channel ELA map.
`TamperLocaliser` (model)	The dual-stream encoder-decoder itself: two independent encoders (RGB+ELA, SRM residual), a fusion layer, and a decoder that outputs a per-pixel tamper logit map. Pure `nn.Module`, no I/O.
`SyntheticTamperDataset`	Generates document-like images with a randomly spliced patch and a pixel-accurate mask, plus rotation/blur/noise/recompression augmentation (NFR-10). Stand-in for FSD (paper [29]) until real scans are available.
`TamperLocalisationService`	The module's only external surface (NFR-7): `predict(image)` runs preprocessing, the model, thresholding, connected-component region extraction and scoring, and returns the fixed output contract. Everything above is an implementation detail behind this one method.

2.2 Internal data flow



2.3 Algorithm detail

Deskew. For angle in [-10°, +10°] stepped by 1°, rotate the grayscale page and score it by the variance of row-wise pixel-intensity sums (text rows create high-variance peaks when horizontal; a skewed page blurs them together). Keep the angle with the highest variance. Cheap, needs no extra dependency, and good enough for the small scan tilts the problem calls for.

ELA. Re-save the page as JPEG at quality 90 and take the per-pixel absolute difference against the original, averaged over the 3 colour channels, normalised to [0, 1]. Untouched regions re-compress almost losslessly; edited regions (pasted from a different source, or re-saved at a different quality) show a brighter residual. Used only as one input channel alongside the RGB stream (Gap 4): the model is free to learn how much to trust it, rather than thresholding on it directly.

SRM kernels (applied per RGB channel, zero-padding, fixed weights, never updated by backprop):

```
horizontal residual   [[ 0,  0, 0], [0, -1, 1], [0,  0, 0]]
vertical residual     [[ 0,  0, 0], [0, -1, 0], [0,  1, 0]]
Laplacian residual    [[ 0,  1, 0], [1, -4, 1], [0,  1, 0]]
```

Loss. `BCEWithLogits(logits, mask) + DiceLoss(logits, mask)`, where `DiceLoss = 1 − (2|P∩T| + ε) / (|P| + |T| + ε)` on sigmoid probabilities `P` against ground truth `T`. BCE alone is dominated by the large background class on a sparsely-tampered page; Dice pulls the gradient toward getting the (small) tampered region right, which is what IoU in the NFR-5 evaluation actually measures.

Region extraction. Threshold the heatmap at 0.5, label connected components with `scipy.ndimage.label`, rank components by pixel area, keep the top 5, and report each as `{bbox: [x, y, w, h], confidence: mean heatmap value inside the component}` — this is the "largest suspicious regions with coordinates" FR-D6 asks for.

Tamper sub-score. `score = round(100 × (0.6 × peak_confidence + 0.4 × coverage_term))`, where `peak_confidence` is the heatmap's maximum value and `coverage_term = min(flagged_area_fraction / 0.2, 1.0)`. A small, high-confidence edit and a large, lower-confidence one can both reach a high score; a few stray low-confidence pixels cannot.

2.4 Interface specification (NFR-7)

Method	`TamperLocalisationService.predict(image: PIL.Image, threshold: float = 0.5) -> dict`
Input	`image`: a full page as a PIL image, any size. `threshold`: mask/region cutoff on the [0,1] heatmap.
Output	`mask`: uint8 array, 256×256, values {0, 255}. `heatmap`: float array, 256×256, [0, 1]. `score`: int, 0–100. `regions`: list of up to 5 `{bbox, confidence}` dicts, largest first.
Construction	`TamperLocalisationService(checkpoint=None, device=None, img_size=256)`. A missing checkpoint runs the (untrained) architecture — useful for interface/contract tests, never for a real score.
Replaceability	The orchestrator only ever calls `predict()`. A newer localiser (different backbone, added anomaly head for Gap 8/O8) is a drop-in replacement for this class as long as it keeps the same method signature and output keys.

2.5 Error handling and edge cases

- **No tamper present.** `regions` is an empty list and `score` is low; this is a normal, expected output, not an error.
- **Degenerate AUC.** If a batch's ground truth is all-genuine or all-forged, `roc_auc_score` is undefined; `evaluate.py` reports `nan` for that batch rather than raising, so a batch imbalance

does not crash an evaluation run (NFR-5).

- **Untrained / missing checkpoint.** `TamperLocalisationService` runs with random weights if no checkpoint is given; callers must not treat that output as a real decision (see Section 2.4, Construction).
- **Device fallback.** `device` defaults to CUDA if available, else CPU, satisfying NFR-2 without any caller-side branching.
- **Oversized/undersized pages.** `preprocess()` resizes to a fixed 256×256 before the model ever sees the image, so input resolution cannot change the model's memory footprint or runtime.

3. Detailed Design — Remaining Services (design-level, not yet implemented)

3.1 Signature Verification Service (FR-S1–S3, O1, O2)

- `SignatureDetector.detect(page) -> bbox`: object detector locating the signature region on the full page (paper [13]).
- `SignatureCleaner.clean(crop) -> crop`: removes ruled lines, stamps and background noise from the detected crop (FR-S2).
- `SignatureEncoder.embed(crop) -> vector`: writer-independent Siamese/metric-learning encoder (papers [3], [6], [12]); frozen at inference.
- `SignatureService.verify(page, signer_id) -> {similarity, decision, bbox}`: runs the three steps above, then compares the embedding against the signer's 1–5 stored reference embeddings by cosine distance and thresholds the result. Adding a signer is an `INSERT` into `signatures`, never a retrain (NFR-6).

3.2 OCR / Field Validation Service (FR-D3, FR-D4)

- `OCREngine.extract(page) -> {field: text}`: text extraction, consensus across engines for financial documents (paper [39]).
- `FieldValidator.check(fields, doc_type) -> {field: pass|fail}`: per-document-type regex/format rules on names, dates and ID numbers (FR-D3).
- `ArithmeticGate.check(fields) -> bool`: for financial documents, `abs(total - (net + tax)) <= tolerance` (FR-D4, paper [39]).

3.3 Fusion & Scoring Service (FR-D5, FR-D6, O5, O6)

```
final_score = w_sig * signature_sub_score
            + w_tmp * tamper_sub_score
            + w_ocr * ocr_sub_score          # weights configurable, sum to 1

label = "Genuine"    if final_score >= 80
      = "Suspicious"  if 40 <= final_score < 80
      = "Forged"      if final_score < 40

reason = template(lowest-scoring sub-check, its threshold, its evidence)
       # e.g. "signature similarity 0.41 (below 0.6 threshold)"
```

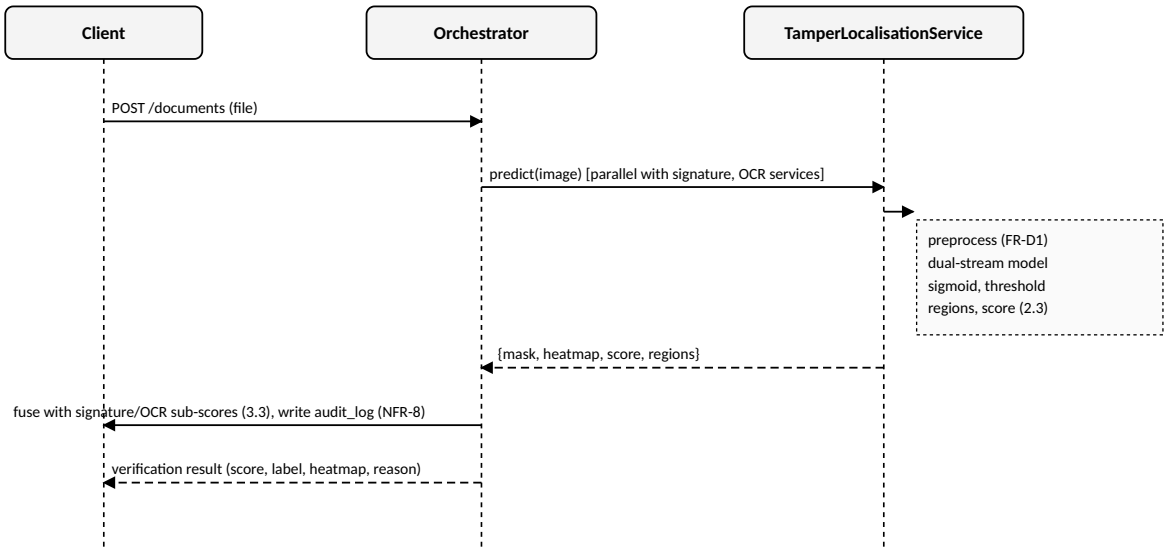
A weighted rule, not a learned end-to-end classifier (Gap 5/O5): each sub-score and the reason it produced stay inspectable, which a black-box classifier would not give (Gap 6/O6).

3.4 Verification Orchestrator (FR-P1)

```
def verify(document):
    pre = preprocess(document)                        # FR-D1
    tamper = tamper_service.predict(pre.image)         # parallel
    signature = signature_service.verify(pre.image, signer_id)
    ocr = ocr_service.check(pre.image, doc_type)
    # fan-in: wait for all three, with a per-service timeout (NFR-1)
    result = fusion_service.score(signature, tamper, ocr)
    audit_log.write(document.id, result)               # NFR-8, append-only
    return result
```

If one service times out or errors, the orchestrator does not fail the whole request: it records that sub-score as unavailable, lowers the confidence of the final label to at most Suspicious, and notes the missing check in the reason text — an unreachable detector degrades the decision rather than silently skipping a check.

4. Verification Sequence (message level)



5. System-Wide Error Handling

- **Invalid file / unsupported type.** Rejected at the API gateway before reaching the orchestrator; never silently coerced into a wrong document type.
- **Detector timeout or failure.** Handled by the orchestrator as in Section 3.4: the request still completes, with a capped confidence and an explicit note in the reason text, rather than a 500 error.
- **Auth / role failure.** API gateway returns 401/403 before any detector runs (NFR-9); nothing is logged to `audit_log` as a verification attempt.
- **Concurrent admin decisions on the same document.** `feedback` inserts are append-only; the latest row is treated as authoritative, and all prior ones remain in the table for audit (NFR-8).

6. Traceability

Section	Traces to
2.1–2.5 (assigned module)	O1–O4, O6–O8; FR-D1, FR-D2, FR-D6; NFR-2, NFR-5, NFR-7, NFR-10; Gap 3, Gap 4
3.1 (signature)	O1, O2; FR-S1–S3; NFR-6; Gap 1, Gap 2
3.2 (OCR/field)	FR-D3, FR-D4; Gap 5
3.3 (fusion)	O5, O6; FR-D5, FR-D6; Gap 5, Gap 6

3.4 (orchestrator)	FR-P1; NFR-1
5 (error handling)	NFR-1, NFR-8, NFR-9

7. Conclusion

The assigned module is now detailed to the level of running code: the component breakdown, the internal data flow diagram and the algorithm descriptions in Section 2 match `tamper_localisation/` line for line. The remaining services are detailed to the same depth in pseudocode, so that implementing them means filling in the classes and functions named here against the same interfaces and data contracts already fixed in the System Design Document.